

Developer API

Place MTN, Telecel and AirtelTigo data-bundle orders from your own system.

One endpoint takes every request. You send a JSON body with an `action` and its fields, authenticated with your API key. Orders are charged to your Bundle Portal wallet, so keep it funded.

POST `https://api.bundleportal.com/v2`

Before you go live: run `verify_number` first. It tells you whether a number is approved for that network and whether a previous order for it is still being delivered. Either would otherwise refuse your order. It is free and creates nothing.

Authentication

Send your key in the `x-api-key` header. Keys start with `bp_live_` and are created from your dashboard under **Developer / API**.

```
x-api-key: bp_live_XXXXXXXXXXXXXXXXXXXXX
Content-Type: application/json
```

Keep your key secret. It can spend your wallet. Use it only from your server, never from a browser or mobile app, where anyone can read it. If a key leaks, generate a new one from your dashboard immediately.

Making a request

Every call is a `POST` to the same URL. The `action` field decides what happens.

```
curl -X POST https://api.bundleportal.com/v2 \
-H "x-api-key: bp_live_XXXXXXXXXXXXXXXXXXXXX" \
-H "Content-Type: application/json" \
-d '{"action":"check_balance"}'
```

```
// Node.js
const res = await fetch('https://api.bundleportal.com/v2', {
  method: 'POST',
  headers: {
    'x-api-key': process.env.BUNDLE_PORTAL_KEY,
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({ action: 'check_balance' }),
});
const data = await res.json();
```

```
// PHP 8+ with cURL
$payload = json_encode(['action' => 'check_balance']);
$ch = curl_init('https://api.bundleportal.com/v2');
curl_setopt_array($ch, [
  CURLOPT_POST => true,
  CURLOPT_RETURNTRANSFER => true,
  CURLOPT_HTTPHEADER => [
    'x-api-key: ' . getenv('BUNDLE_PORTAL_KEY'),
    'Content-Type: application/json',
  ],
  CURLOPT_POSTFIELDS => $payload,
]);
$data = json_decode(curl_exec($ch), true);
$status = curl_getinfo($ch, CURLINFO_HTTP_CODE);
curl_close($ch);
```

Rate limits

Requests are rate limited per API key. Reads and order placement have separate allowances, and both are set well above what a correctly written integration needs.

API v2 has **no status polling**. Order results are sent to your webhook the moment an order settles, so there is nothing to poll. See Order status.

A 429 carries both a `Retry-After` header and `retry_after` seconds in the JSON body. Wait that long, then back off with jitter. Use bounded concurrency, and never retry a spending request without the same `order_id`. Limits may be adjusted without notice; treat the response, not a fixed number, as the source of truth.

Verify a number free

Checks a number before you commit. No order is created and nothing is charged.

```
{
  "action": "verify_number",
  "network": "mtn",
  "recipient": "0241234567"
}
```

```
{
  "success": true,
  "data": {
    "network": "mtn",
    "recipient": "0241234567",
    "allowed": true,
    "can_order": true,
    "allowlist_message": null,
    "pending_order": null
  }
}
```

FIELD	MEANING
allowed	The number is approved for this network.
can_order	Approved <i>and</i> no earlier order is still in flight. Only order when this is <code>true</code> .
pending_order	Details of the unfinished order blocking this number, or <code>null</code> .

List bundles

Returns the packages available to you, at your prices.

```
{ "action": "get_bundles", "network": "mtn" }
```

`network` is optional. Omit it for everything. Each bundle carries an `id` , `size` , `size_gb` , `price` and `validity` . Use `size_gb` as the `package_size` when ordering.

```
{
  "success": true,
  "data": {
    "bundles": [{
      "id": 42, "network": "mtn", "size": "5GB",
      "size_gb": 5, "price": 20.00, "validity": "30 days",
      "pricing_source": "user_override", "has_custom_price": true,
      "has_role_price": false
    }],
    "count": 1
  }
}
```

Place an order

POST action: place_order

```
{
  "action": "place_order",
  "network": "mtn",
  "recipient": "0241234567",
  "package_size": 5,
  "order_id": "your-own-reference-001"
}
```

FIELD	REQUIRED	NOTES
network	yes	mtn , mtn_2 , mtn_3 , telecel , airteltigo (or ishare). See MTN options below.
recipient	yes	10 digits starting 0 , e.g. 0241234567
package_size	yes	Size in GB, matching a bundle from <code>get_bundles</code>
order_id	no	Your reference. Letters, numbers, _ and - , up to 80 characters. Strongly recommended, see below.

MTN options

MTN is available on three independent delivery routes. They are separate catalogues, so call `get_bundles` with the same network value you intend to order on, because sizes and prices can differ per route.

VALUE	USE
mtn	Default. Use this unless you have a reason not to. Existing integrations already send it and need no change.
mtn_2	Alternative MTN route. Availability can vary by recipient number.
mtn_3	Alternative MTN route, with a more limited range of sizes.

mtn_1 is accepted as a legacy alias of mtn and behaves identically, so older integrations keep working unchanged.

```
{
  "success": true,
  "message": "Order placed successfully",
  "data": {
    "order_id": "your-own-reference-001",
    "reference": "KT-88213",
    "network": "mtn",
    "recipient": "0241234567",
    "bundle": "5GB",
    "amount": 20.00,
    "status": "processing",
    "new_balance": 180.00
  }
}
```

A status of `cached` means the order was accepted and queued for manual delivery rather than sent straight to a provider. It is still a real, paid order.

Idempotency

Always send your own `order_id` . If a reply is lost and you retry with the same `order_id` , you get the original order back with `"duplicate": true` , so you are not charged twice. Without one, a retry is a brand new order.

Only one order can be in flight per recipient number at a time. A second order for a number still being delivered is refused with `409 pending_order` . This protects your customers from paying twice for the same bundle.

Order status

API v2 does not support status polling. When an order is delivered, fails, is cancelled or refunded, we send the result to your webhook straight away, so you never need to ask. Every

order is also visible in your Bundle Portal dashboard.

A `check_status` request is refused:

```
HTTP 410
{
  "success": false,
  "code": "polling_disabled",
  "message": "Order status is not polled on v2. When an order is delivered or fails we send the result to your webhook."
}
```

The `status` field in the webhook body uses these values:

STATUS	MEANING
processing	Sent to the provider, delivery under way.
cached	Accepted and queued for manual delivery.
completed	Delivered.
failed	Not delivered. Any charge is reversed.

Wallet balance

```
{ "action": "check_balance" }
```

```
{
  "success": true,
  "data": {
    "wallet_balance": 180.00, "currency": "GHS",
    "user": { "name": "Ama Mensah", "email": "ama@example.com" }
  }
}
```

Transactions

```
{ "action": "get_transactions", "limit": 20, "offset": 0 }
```

```
{
  "success": true,
  "data": {
    "transactions": [{
      "order_id": "your-own-reference-001", "type": "purchase",
      "network": "mtn", "bundle": "5GB", "amount": 20.00,
      "phone_number": "0241234567", "status": "completed",
      "created_at": "2026-08-15T10:20:00.000Z"
    }], "count": 1, "limit": 20, "offset": 0
  }
}
```

Result checkers

Buy WASSCE and BECE checkers. Each purchase returns one serial and PIN.

```
{ "action": "get_result_checkers" }
```

```
{
  "success": true,
  "data": { "result_checkers": [{
    "type": "wassce", "name": "WASSCE Result Checker",
    "price": 18.00, "available": 35, "in_stock": true
  }] }
}
```

```
{ "action": "buy_result_checker", "product_type": "wassce" }
```

```
{
  "success": true, "message": "Result checker purchased",
  "data": {
    "success": true, "name": "WASSCE Result Checker",
    "serial": "WR123456789", "pin": "123456789012",
    "amount": 18.00, "wallet_balance": 162.00, "transaction_id": 9182
  }
}
```

Each product reports `available` stock and whether it is `in_stock` . If the feature has been switched off you receive `503 feature_disabled` .

Special offers

```
{ "action": "get_special_offers" }
```

```
{
  "success": true,
  "data": { "special_offers": [{
    "id": 3, "title": "Weekend Data Offer",
    "description": "Promotional bundle", "price": 12.00
  }] }
}
```

```
{ "action": "buy_special_offer", "offer_id": 3, "recipient": "0241234567" }
```

```
{
  "success": true, "message": "Offer purchased, queued for processing",
  "data": {
    "order_id": "OFFER_1786781000_AB12CD", "transaction_id": 9183,
    "offer": "Weekend Data Offer", "recipient": "0241234567",
    "amount": 12.00, "status": "processing", "new_balance": 150.00
  }
}
```

Airtime

Top up any MTN, Telecel or AirtelTigo number. The recipient always receives the *face value* you ask for; what leaves your wallet is the face value adjusted by your account's rate, so read `pay_ratio` from the config call rather than assuming the two are the same.

```
{ "action": "get_airtime_config" }
```

```
{
  "success": true,
  "data": {
    "enabled": true, "minAmount": 0.50, "maxAmount": 500.00,
    "networks": ["mtn", "telecel", "airteltigo"],
    "payRatio": 0.965, "balance": 150.00
  }
}
```

`payRatio` is what you pay per GH¢1 of face value. At `0.965`, GH¢10 of airtime costs you GH¢9.65 and the recipient still receives GH¢10.00.

```
{ "action": "place_airtime", "network": "mtn",  
  "recipient": "0241234567", "amount": 10 }
```

```
{  
  "success": true,  
  "data": {  
    "order_id": "AIRX7QAG6VR", "status": "processing",  
    "amount": 9.65, "face_value": 10.00,  
    "phone_number": "0241234567", "balance": 140.35  
  }  
}
```

`amount` is what you were charged; `face_value` is what the recipient received. The result is sent to your webhook. A top-up the provider explicitly rejects is refunded automatically; one it never answers is left for review and is **not** auto-refunded, because an unanswered order may still have been delivered.

Data bundles

A second catalogue, separate from List bundles: non-expiry, midnight, social, video and call-minute packages. Prices already include your account's rate, so charge what `sell_price` says.

```
{ "action": "get_data_bundles" }
```

```
{  
  "success": true,  
  "data": {  
    "enabled": true, "balance": 150.00, "flexiPayRatio": 0.965,  
    "packages": [{  
      "id": 42, "display_name": "No Expiry · 1GB",  
      "group_label": "No Expiry", "network": "telecel",  
      "data_amount": "1GB", "sell_price": 9.65,  
      "is_flexi": false, "min_amount": null, "max_amount": null  
    }]  
  }  
}
```

A package with `is_flexi: true` has no fixed price. You name any amount between `min_amount` and `max_amount`, and it costs that amount multiplied by `flexiPayRatio`.

```
{ "action": "place_data_order", "package_id": 42,
  "recipient": "0201234567" }
```

```
{
  "success": true,
  "data": {
    "order_id": "DATE0WJVH5S", "status": "processing",
    "amount": 9.65, "phone_number": "0201234567",
    "balance": 140.35
  }
}
```

Send `amount` as well for a flexi package. Ordering the same package to the same number twice within three minutes returns `409` rather than sending it twice.

Bill providers read-only

Lists the electricity, TV, water and internet billers we carry. Paying a bill is **not** offered on the API yet, so this is here to build against.

```
{ "action": "get_bill_providers" }
```

```
{
  "success": true,
  "data": { "providers": [{
    "provider_code": "ECG", "name": "ECG Prepaid",
    "category": "Electricity", "account_label": "Meter number",
    "min_amount": 5.00, "max_amount": 1000.00
  }] }
}
```

Share data

Sends data from a share balance the admin pre-loads on your account, not from your wallet.

```
{ "action": "get_ishare_balance" }
{ "action": "share_ishare", "recipient": "0271234567", "mb": 500 }

{ "action": "get_telecel_balance" }
{ "action": "share_telecel", "recipient": "0201234567", "gb": 2 }
```

Balance responses use the same shape for both networks:

```
{
  "success": true,
  "data": { "network": "iShare", "balance": 4500, "unit": "MB", "enabled": true }
}
```

Share responses use the same shape for `share_ishare` and `share_telecel` :

```
{
  "success": true, "message": "iShare data is on its way",
  "data": {
    "order_id": "ISHARE_1786781000_AB12", "network": "iShare",
    "recipient": "0271234567", "amount": 500, "unit": "MB",
    "status": "processing", "balance_remaining": 4000,
    "timestamp": "2026-08-15T10:35:00.000Z"
  }
}
```

Send an `order_id` with share requests too. A repeated ID returns the original share with `duplicate: true` and does not deduct the share balance again.

Webhooks required

Register a URL once and we call it the moment an order reaches a final state. On API v2 this is how you learn the outcome of every order. There is no status polling.

```
{ "action": "set_webhook", "webhook_url": "https://your-server.example/bundleportal/orders" }
```

```
{
  "success": true,
  "data": {
    "webhook_url": "https://your-server.example/bundleportal/orders",
    "webhook_secret": "whsec_4f1c...",
    "events": ["order.completed", "order.failed", "order.cancelled", "order.refunded"]
  }
}
```

The secret is shown once. Store it when you create the webhook. It cannot be read back. Call `set_webhook` again to rotate it, or `delete_webhook` to stop delivery. `get_webhook` returns the registered URL but never the secret.

The URL must be `https` and must resolve to a public host. We POST this body:

```
{
  "event": "order.completed",
  "order_id": "your-own-reference-001",
  "reference": "KT-88213",
  "status": "completed",
  "network": "mtn",
  "bundle": "5GB",
  "recipient": "0244000000",
  "amount": 24.5,
  "failure_reason": null,
  "settled_at": "2026-08-15T10:22:04.000Z"
}
```

Verify every callback. The `X-BundlePortal-Signature` header is `sha256=<hex>`, an HMAC-SHA256 of the *raw request body* keyed with your webhook secret. Compare it using a constant-time comparison, and reject anything that does not match. Your endpoint is public, so the signature is what distinguishes our call from anyone else's.

```
// Node
const expected = 'sha256=' + crypto.createHmac('sha256', secret).update(rawBody).digest('hex');
const ok = crypto.timingSafeEqual(Buffer.from(expected), Buffer.from(header));
```

Answer `2xx` quickly and do the work afterwards. Deliveries have a 5 second timeout and are not retried, so keep your endpoint available and fast. If you ever miss one, the order and its final status are in your Bundle Portal dashboard.

Errors

Failures return `success: false` with a `message` safe to show a customer, and usually a `code` you can branch on.

```
{
  "success": false,
  "error": "Recipient not approved",
  "code": "not_allowlisted",
  "message": "This number is awaiting approval for MTN."
}
```

HTTP / IDENTIFIER	CAUSE	FIX
400 · validation	Missing <code>action</code> , invalid JSON, network, phone, size, reference, product, or offer.	Correct the field named by <code>message</code> . Do not retry unchanged input.
400 · provider/product failure	No matching bundle, recipient ineligible, result checker unavailable/out of stock, or provider rejected the order.	Refresh products with the matching list action, confirm eligibility, or show the returned message. The order is not left charged after a hard provider rejection.
401 · authentication	The <code>x-api-key</code> header is missing, malformed, revoked, or unknown.	Use the current <code>bp_live_...</code> key from Developer / API. Regenerate it if compromised.
402 · balance changed	The authoritative wallet/share balance was too low when the atomic charge ran.	Call <code>check_balance</code> , top up, then retry with the same <code>order_id</code> .
403 · not_allowlisted	The number is not yet approved for that network.	Run <code>verify_number</code> and retry only after <code>allowed</code> becomes <code>true</code> .
403 · channel_locked	All bundle ordering or the Developer API channel is paused for maintenance.	Stop sending spending requests and retry later; balance and status checks remain safe.
403 · role_locked	Ordering is paused for the account's API tier.	Contact support; retries will not help until the tier is unlocked.
403 · paused / unavailable	Transactions are paused or an account safety limit has been reached.	Show <code>message</code> . For a stated daily limit, wait until the next day or contact support.
403 · suspended	API access for this account has been suspended.	Contact support.
409 · pending_order	An earlier order for the same number is still processing or cached.	Wait for that order's webhook; place the next order only after it reaches a final state.

410 · polling_disabled	check_status was called. API v2 has no status polling.	Register a webhook; results are sent to it automatically.
409 · network_locked	The network is temporarily out of stock. out_of_stock is true .	Do not switch network labels for the same number. Wait for stock to return.
429 · rate limit	The key exceeded its current per-minute request allowance.	Wait the Retry-After / retry_after interval, then back off with jitter.
429 · read_rate_limited	Too many lookup calls on this key. Order placement is unaffected.	Slow down lookups such as get_bundles and check_balance ; cache their results.
500 · server error	An unexpected database, wallet, or status lookup failure occurred.	Retry a read. Retry a purchase only with the exact same order_id ; contact support if it persists.
503 · order_capacity_busy	The service is briefly at capacity for provider- bound orders.	Wait the Retry-After interval, then retry with the exact same order_id .
503 · feature_disabled / unavailable	Result checkers or data sharing is switched off.	Hide that product temporarily and retry its list/balance action later.

Handling failures: treat 409 and 429 as retry-later, not as permanent errors. Treat 403 not_allowlisted as "pending approval" and tell your customer so, rather than showing a generic failure.

Support

Email bundleportal1@gmail.com with your order_id and the response you received.

Bundle Portal Developer API. Generated from the live documentation at bundleportal.com/api-docs